

TECHNICAL RESEARCH REPORT

ThreatInt

An Automated Threat-Intelligence Pipeline

Collection · Cross-Verification · ML Classification · Campaign Clustering

Repository: `Ashwin-kumar-jonjale/ThreatInt`

Branch: `feat/threat-intel-pipeline`

Artifact: `threatint` v0.1.0 · MIT licence

Stack: Python 3.10+ · scikit-learn · Flask · Docker · GitHub Actions

Every quantitative figure in this report is derived from a real, deterministic offline run of the pipeline (seed 42). No numbers are illustrative.

Compiled with L^AT_EX · October 6, 2026

Contents

1	Executive Summary	1
2	Introduction and Problem Statement	1
2.1	The analyst bottleneck	1
2.2	What the system must do	1
2.3	Scope and non-goals	2
3	Design Goals and Principles	2
4	System Architecture	2
4.1	The pipeline	2
5	The Data Model	4
6	Module-by-Module Reference	4
6.1	normalize.py — canonicalisation and extraction	5
6.2	config.py — typed configuration	5
6.3	collectors/base.py — source ingestion	5
6.4	verification/__init__.py — trust and consensus	6
6.4.1	Source reliability, in full	6
6.4.2	Indicator scoring	7
6.5	features/__init__.py — feature engineering	7
6.6	ml/ — classification	7
6.7	clustering/__init__.py — campaign discovery	8
6.8	pipeline.py, reporting.py, cli.py	9
6.9	webapp/ — the presentation layer	9
7	Methodology Summary	10
8	Experimental Setup and Results	10
8.1	Setup	10
8.2	Composition	11
8.3	Source reliability	11
8.4	Classifier performance	11
8.5	Campaign clustering	13
8.6	Per-feed contribution	14
9	Interfaces, Deployment and Reproducibility	14
9.1	Command-line interface	14
9.2	Web dashboard and static export	15
9.3	Deployment paths	15
9.4	Testing	15
10	Security Considerations	15
11	Limitations	16
12	Future Work	16
13	Conclusion	16

A	Configuration Reference	17
B	CLI Reference	17
C	Reproducing This Report	17

1 Executive Summary

Security operations centres are drowning in alerts. A single analyst may be expected to cross-check hundreds of indicators-of-compromise (IOCs) per day against a dozen overlapping feeds, reconcile their disagreements by hand, and decide what is genuinely malicious. The work is repetitive, slow, and error-prone: the same indicator appears in five feeds with five different labels, and the analyst’s real value — judgement and context — is spent on mechanical reconciliation.

ThreatInt is a complete, reproducible pipeline that automates that mechanical layer end to end. It collects indicators from multiple trusted sources, normalises and deduplicates them, scores the *reliability of each source* against the cross-source consensus, classifies each indicator with a supervised machine-learning model, and clusters the malicious ones into *attack campaigns* using unsupervised density-based clustering. The output is an analyst-facing console report, machine-readable JSON/CSV, and an interactive web dashboard.

Headline results — offline reference run

106 raw records ingested from 5 feeds	81 unique indicators after dedup
13 indicators corroborated by ≥ 2 sources	57 classified malicious, 24 benign
8 attack campaigns discovered	classifier acc 0.860 · F1 0.850 · AUC 0.946

The design makes three deliberate bets that distinguish it from a naive feed aggregator:

1. **Trust is computed, not assumed.** Each feed’s static analyst prior is blended with dynamic signals (precision, recall, pairwise agreement, volume) measured against the consensus. An indicator’s score is then the reliability-weighted vote of the feeds reporting it, with a saturating bonus for independent corroboration.
2. **The pipeline is reproducible and network-optional.** Every collector supports a bundled fixture path and degrades gracefully to it on any fetch failure. The whole system runs offline, deterministically, in CI — which is what makes the evaluation in this report trustworthy.
3. **The ML is interpretable.** Features are human-legible (entropy, suspicious-TLD flag, phishing-keyword count, port risk), so the model’s decisions can be explained to an analyst rather than black-boxed.

2 Introduction and Problem Statement

2.1 The analyst bottleneck

Threat intelligence arrives as a firehose of partially-overlapping feeds: domain blocklists, URL blacklists, phishing registries, honeypot captures, malware-family trackers. Three structural problems make manual triage expensive:

Volume Thousands of new IOCs per day across feeds, far exceeding human review capacity.

Disagreement Feeds overlap inconsistently. The same domain may be flagged by one feed and ignored by four others, with no obvious way to weigh whose judgement matters.

Fragmentation Related indicators arrive as scattered, unconnected rows. A single campaign — say, a phishing kit using five lookalike domains and a shared C2 IP — is invisible unless something groups it.

2.2 What the system must do

ThreatInt targets exactly these three problems with three corresponding capabilities, wrapped in one deterministic pipeline:

Problem	Capability
Volume	Automated collection from multiple feeds, normalised and deduplicated
Disagreement	Cross-verification: per-source reliability + consensus scoring
Fragmentation	ML classification + DBSCAN campaign clustering

2.3 Scope and non-goals

ThreatInt is an *aggregation, scoring and grouping* system. It deliberately does not fetch live malware, perform active scanning, or make blocking decisions. Its outputs are advisory: ranked indicators, confidence scores, and campaign groupings for a human to act on. This keeps the blast radius small and the system auditable.

3 Design Goals and Principles

The codebase is small (2,246 lines of Python across 18 modules) but the design decisions behind it are load-bearing. Six principles shaped it:

1. **Plain data structures over frameworks.** The core entities are `dataclasses` (`RawRecord`, `Observation`, `Indicator`, `Campaign`, `SourceReliability`), not ORM or framework objects. Stages communicate by passing lists of these; nothing shares mutable global state. This makes every stage independently testable and reorderable.
2. **Network-optional by construction.** A collector’s contract is “return `RawRecords`, never assume the network.” Every source has a bundled fixture, and `HttpCollector` falls back to it on *any* exception. Offline runs are the default for tests and the dashboard.
3. **Determinism.** A single `random_seed` (42) governs corpus generation, the train/test split, and the classifier. The same run yields the same numbers — the precondition for the evaluation here.
4. **Separation of collection from interpretation.** Collectors emit source-specific records; a dedicated normalisation layer canonicalises them. A new feed is added by writing one parser, not by touching the ML.
5. **Interpretability first.** Features and source scores are chosen so a human can read them. The model is a random forest with Gini importances, not an opaque embedder.
6. **Fail soft, never fail the run.** A single bad feed must not abort the pipeline. Collection is wrapped per-source; model persistence is best-effort; the API clamps its inputs.

4 System Architecture

ThreatInt is organised as four horizontal layers sitting on a shared data model, with three entry points on top. The architecture figure below shows the whole system at a glance.

4.1 The pipeline

Orchestration lives in `pipeline.run_pipeline`, which executes seven ordered stages. Each stage is a small function owning no global state, so a run is fully described by its `PipelineResult`. Figure 2 annotates the stages with live counts from the reference run.

The stage sequence is:

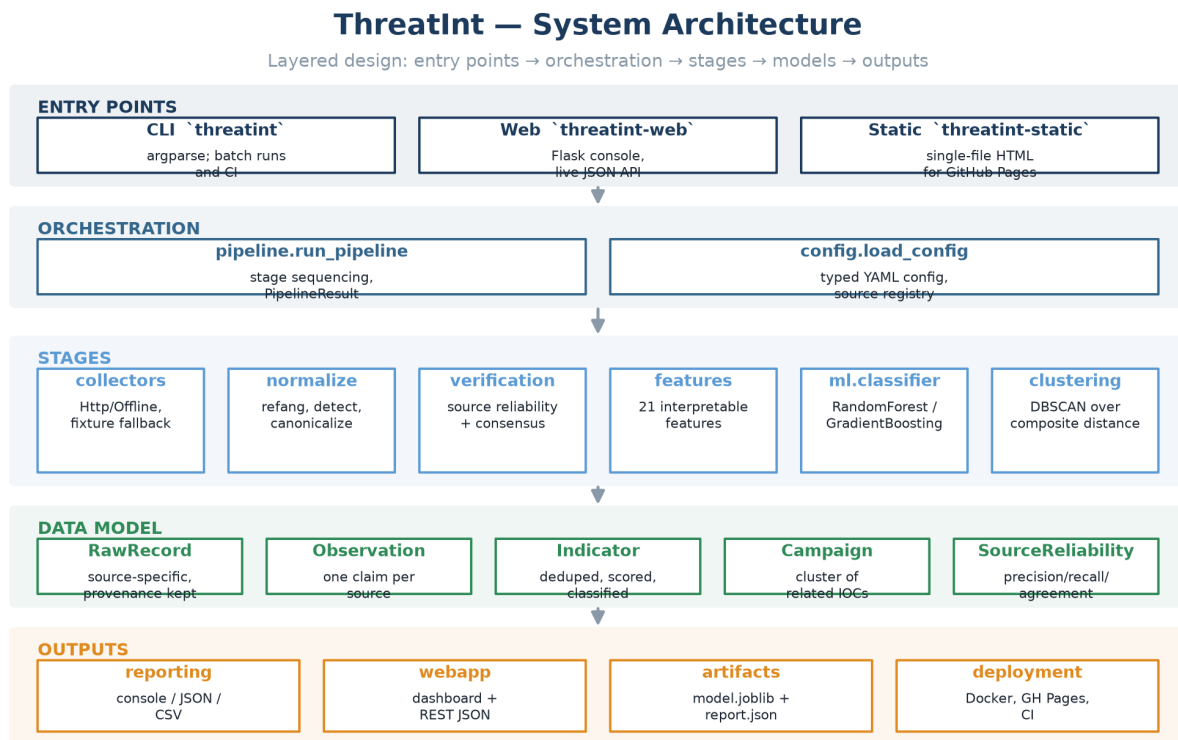


Figure 1: System architecture. Three entry points (CLI, live web console, static export) drive a shared orchestrator, which sequences six pipeline stages over a common data model and emits reports, a dashboard and persisted artifacts.

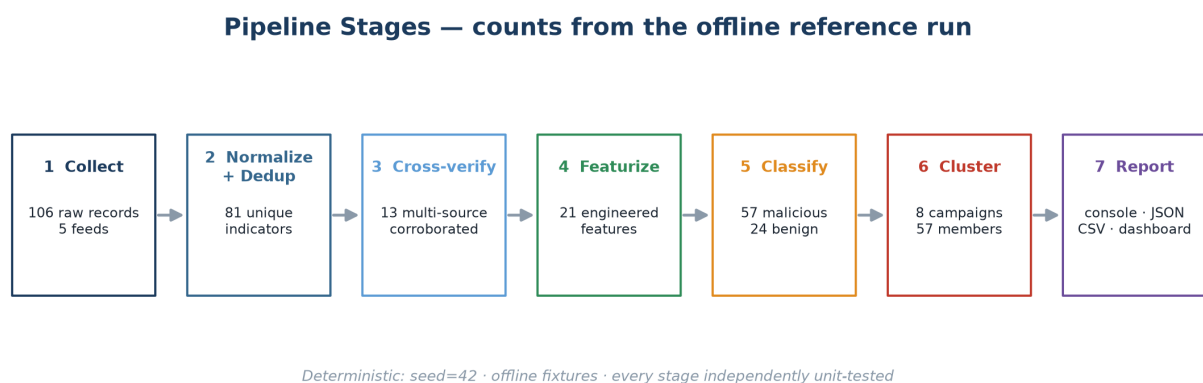


Figure 2: The seven pipeline stages with counts from the offline reference run. Collect and verify collapse raw records into deduplicated, reliability-scored indicators; featurize/classify attach verdicts; cluster groups the malicious ones into campaigns.

Stage	Module	Responsibility
Collect	collectors/	Instantiate a collector per enabled source; emit <code>RawRecords</code> ; degrade to fixtures on failure.
Verify	verification/	Score each source's reliability; collapse observations into scored, deduplicated <code>Indicators</code> .
Featurize	features/	Attach a fixed 21-dimensional numeric vector to every indicator.
Classify	ml/	Train (or load) the classifier; set <code>malicious_probability</code> and <code>verdict</code> .
Cluster	clustering/	DBSCAN over a composite distance; produce <code>Campaigns</code> and stamp members.
Report	reporting/	Console, JSON and CSV renderings.
Present	webapp/	Flask dashboard and static HTML export.

5 The Data Model

Five dataclasses in `models.py` define every shape that moves through the system. The pipeline is essentially a sequence of transforms between them: `RawRecord` $\rightarrow$ `Observation` $\rightarrow$ `Indicator` $\rightarrow$ `Campaign`.

Type	Stage	Key fields and meaning
<code>RawRecord</code>	collect	<code>source</code> , <code>indicator</code> , <code>indicator_type</code> , <code>observed_at</code> , <code>tags</code> , <code>confidence</code> , <code>raw</code> . The raw dict preserves the original payload for provenance and re-runs.
<code>Observation</code>	verify	One source's claim: <code>source</code> , <code>indicator</code> , <code>indicator_type</code> , <code>observed_at</code> , <code>confidence</code> , <code>tags</code> . Many observations collapse to one indicator.
<code>Indicator</code>	verify-cluster	Deduplicated observable. <code>indicator_id</code> is a stable hash of (type, value); carries <code>source_count</code> , <code>reliability_score</code> , <code>verdict</code> , <code>malicious_probability</code> , <code>campaign_id</code> , <code>features</code> .
<code>SourceReliability</code>	verify	<code>prior</code> , <code>precision</code> , <code>recall</code> , <code>agreement</code> , <code>volume</code> , <code>score</code> .
<code>Campaign</code>	cluster	<code>campaign_id</code> , <code>label</code> , <code>indicators</code> , <code>shared_tags</code> , with derived <code>size</code> , <code>mean_reliability</code> , <code>mean_malicious_probability</code> .

Why `indicator_id` is a hash

The identifier is `sha256("type:value")[:16]`. Because it is a pure function of the canonical type and value, the *same* observable reported by five feeds collapses to a single object with no coordination between collectors. It is also stable across runs, so campaign IDs — themselves derived from member IDs — are reproducible.

Two enumerations anchor the vocabulary: `IndicatorType` (`ipv4`, `domain`, `url`, `md5`, `sha256`) and `Verdict` (`malicious`, `benign`, `unknown`).

6 Module-by-Module Reference

This section walks the codebase in execution order. Each module's public functions are tabulated with their purpose and behaviour.

6.1 `normalize.py` — canonicalisation and extraction

Threat feeds are messy: defanged notation (`hxxp://, 1[.]2[.]3[.]4`), mixed types in one column, URLs carrying credentials, inconsistent casing. This module turns any of that into validated, canonical indicators.

Function	Purpose
<code>refang(value)</code>	Reverse common defanging (<code>hxxp</code> → <code>http</code> , <code>[.]</code> → <code>.</code>) so indicators can be parsed.
<code>defang(value)</code>	Safe-to-share rendering; used by the dashboard so live IOCs are never shown clickable.
<code>canonicalize(value, type)</code>	Return the canonical form or <code>None</code> . Lower-cases domains/hashes, strips ports off IPs, rebuilds URLs from parsed components, drops fragments.
<code>detect_type(value)</code>	Best-effort inference of an indicator's type via ordered regex checks (<code>URL</code> → <code>IP</code> → <code>hash</code> → <code>domain</code>).
<code>extract_indicators(text)</code>	Pull every indicator out of a free-text blob; URLs are matched first and their spans excluded so a URL's host is not double-counted as a bare domain.
<code>is_public_ip(value)</code>	True for globally-routable IPv4 (excludes private, loopback, link-local, reserved).

Validation is strict and specific: `_is_valid_domain` enforces label length and character rules and rejects a benign-domain/TLD denylist (`localhost`, `example.com`, `invalid`, `.test`); hashes must be exactly 32 or 64 hex characters; URLs must parse with an allowed scheme and a valid host.

6.2 `config.py` — typed configuration

Configuration is a YAML file loaded into typed dataclasses, with path resolution that works both from the repo layout and from an installed wheel.

Function / type	Purpose
<code>load_config(path)</code>	Load and validate YAML into <code>Config</code> ; raises on a non-mapping root or a source entry missing <code>name/kind</code> .
<code>_resolve_config_path</code>	Resolution order: explicit arg → <code>\$THREATINT_CONFIG</code> → repo path → packaged copy.
<code>Config</code>	Holds <code>pipeline</code> , <code>sources</code> , <code>features</code> , <code>ml</code> , <code>clustering</code> , <code>data_dir</code> ; exposes <code>enabled_sources</code> and <code>get_source(name)</code> .
<code>SourceConfig</code>	One feed: <code>name</code> , <code>kind</code> , <code>reliability_prior</code> , <code>url</code> , <code>fixture</code> , <code>api_key_env</code> , <code>enabled</code> .
<code>PipelineConfig</code>	<code>min_sources_for_verification</code> , <code>malicious_threshold</code> , <code>min_reliability_for_clustering</code> , <code>random_seed</code> .

6.3 `collectors/base.py` — source ingestion

A collector's only job is to turn a source into `RawRecords`. The base class provides parsing helpers; two concrete collectors cover the two modes.

Function / class	Purpose
<code>_parse_ts(value)</code>	Tolerant timestamp parsing across five formats plus ISO-8601, defaulting to “now” (UTC) on failure.
<code>_split_tags(value)</code>	Normalise tag fields split on ;, or , into a lower-cased list.
<code>BaseCollector._make_record</code>	Refang, detect type, drop unusable values, and build a <code>RawRecord</code> with confidence defaulting to the source prior.
<code>BaseCollector._read_fixture</code>	Parse a generic fixture CSV with case-insensitive, order-independent columns.
<code>OfflineCollector.fetch</code>	Read the bundled fixture; used for <code>-offline</code> and for sources with no URL.
<code>HttpCollector.fetch</code>	Fetch a remote feed, optionally with a bearer token; on <i>any</i> exception fall back to the fixture.
<code>HttpCollector._parse_text</code>	Parse tabular CSV, or if the payload is not tabular, extract every indicator from the raw text.
<code>build_collectors(config, offline)</code>	Instantiate the right collector per enabled source.

The fallback is a feature — not a hack

A flaky or rate-limited feed should degrade to last-known-good local data rather than abort the run. `HttpCollector` catches `Exception` broadly and returns the fixture, logging a warning. In a live validation run, URLhaus returned 32,289 real records while an AlienVault OTX endpoint returned HTTP 403 — and the pipeline completed cleanly, mixing live and fixture data.

6.4 verification/_init_.py — trust and consensus

This is the analytical heart of the system. It answers two questions: how much to trust each *feed*, and how much to trust each *indicator*.

Function	Purpose
<code>_jaccard(a, b)</code>	Set-overlap similarity $ a \cap b / a \cup b $; the basis of pairwise feed agreement.
<code>compute_source_reliability</code>	For every source, compute precision, recall and mean pairwise agreement against the consensus set, then blend with the prior.
<code>score_indicator</code>	Reliability-weighted average of the reporting sources’ confidences, plus a saturating consensus bonus.
<code>build_indicators</code>	Group observations by (type, value); construct one <code>Indicator</code> per group with first/last-seen, merged tags, source count and score; sort by reliability.

6.4.1 Source reliability, in full

Let S be the set of sources. For source s with reported indicator set R_s , let the consensus set be

$$C = \{x : |\{s : x \in R_s\}| \geq m\}, \quad m = \text{min_sources_for_verification}.$$

Then

$$\text{precision}_s = \frac{|R_s \cap C|}{|R_s|}, \quad \text{recall}_s = \frac{|R_s \cap C|}{|C|}, \quad \text{agreement}_s = \frac{1}{|S| - 1} \sum_{t \neq s} J(R_s, R_t),$$

and the dynamic component blends them as

$$d_s = 0.45 \text{precision}_s + 0.25 \text{recall}_s + 0.30 \text{agreement}_s.$$

A volume factor $v_s = 1 - e^{-|R_s|/25}$ damps sources with too little evidence, and the final score is

$$\text{score}_s = \underbrace{p_s(1 - v_s)}_{\text{prior dominates}} + \underbrace{\left(\frac{1}{2}p_s + \frac{1}{2}d_s\right)v_s}_{\text{evidence dominates}}, \quad p_s = \text{analyst prior}.$$

Reading the formula

A feed with almost no data keeps its analyst prior. A feed with plenty of data is judged mostly on measured behaviour — but never fully, because the prior keeps a 50% share even at full volume. This prevents a single noisy batch from destroying a trusted feed’s standing.

6.4.2 Indicator scoring

For an indicator reported by observations o with source scores score_s and confidences c_o ,

$$\text{weighted} = \frac{\sum_o \max(0.01, \text{score}_s) c_o}{\sum_o \max(0.01, \text{score}_s)}, \quad \text{bonus} = 0.15 \min\left(1, \frac{n-m+1}{3}\right),$$

where n is the number of *distinct* sources. The bonus saturates at 0.15, so a feed reported by twenty sources is not treated as twenty times more credible than one reported by three.

6.5 features/__init__.py — feature engineering

A single fixed-order 21-dimensional vector is produced per indicator and feeds *both* the classifier and the clustering stage, guaranteeing they see the same representation.

Function / class	Purpose
<code>_entropy(text)</code>	Shannon entropy of the character distribution; high values flag DGA-like strings.
<code>_ratio(text, pred)</code>	Fraction of characters satisfying a predicate (digit/alpha ratio).
<code>FeatureExtractor.extract</code>	Compute the full 21-feature dict for one indicator.
<code>FeatureExtractor.transform</code>	Return a row-major numeric matrix for a sequence of indicators.
<code>featurize(indicators, config)</code>	Attach features in place.

The 21 features span three families: *type flags* (`is_ipv4/is_domain/is_url/is_hash`), *lexical structure* (`length, digit_ratio, alpha_ratio, entropy, dot_count, hyphen_count`), and *suspicion signals* (`has_suspicious_tld, suspicious_keyword_count, port_high_risk, is_public_ip`) plus the cross-source trust signals (`source_count, reliability_score, tag_count, age_days`).

6.6 ml/ — classification

The classifier is a scikit-learn estimator behind a stable feature contract, with two training-data paths and auditable persistence.

Function / class	Purpose
<code>_make_estimator(cfg, seed)</code>	Build a <code>RandomForestClassifier</code> (default) or <code>GradientBoostingClassifier</code> ; RF uses <code>class_weight="balanced"</code> .
<code>load_labeled_data(path)</code>	Load analyst labels from a CSV with <code>indicator/label</code> (alias-aware) columns.
<code>IndicatorClassifier.train</code>	Build usable pairs, split (stratified, seeded), fit, and compute accuracy/precision/recall/F1/AUC plus feature importances.
<code>IndicatorClassifier.predict</code>	Set <code>malicious_probability</code> and derive the verdict from the threshold.
<code>IndicatorClassifier.save / load</code>	Persist/restore the estimator plus its training report via <code>joblib</code> ; a <code>.report.json</code> sidecar is written for audit.
<code>generate_corpus(n, seed)</code>	Deterministic synthetic bootstrap corpus with separable malicious/benign distributions.

Verdicts are thresholded as: **malicious** if $p \geq \theta$; **benign** if $p \leq 1 - \theta$; otherwise **unknown** — so the classifier abstains in the ambiguous band rather than forcing a binary call.

Synthetic data is clearly labelled

The default training data is a *seeded synthetic bootstrap corpus* whose generator encodes the same lexical intuitions an analyst uses (DGA-like strings, phishing keywords, abused TLDs, high-risk ports). It exists so the plumbing is exercised end to end; it is always replaced the moment `-labels` points at analyst-reviewed data, and every report records `data_source` so a synthetic-trained model is never mistaken for a production one.

6.7 clustering/_init_.py — campaign discovery

Related indicators are grouped with DBSCAN over a *composite* distance blending lexical and semantic similarity.

Function	Purpose
<code>_vector(indicator)</code>	Weighted feature vector; strong semantic signals (type, suspicious tokens) are up-weighted, raw length is damped.
<code>_composite_distance</code>	Pairwise distance in $[0, 1]$: $0.6 \cdot \text{lexical cosine} + 0.4 \cdot \text{tag-set disagreement}$.
<code>_campaign_id(members)</code>	Stable campaign identifier: <code>"camp-" + sha256(sorted member IDs)[:10]</code> .
<code>_label_for(members)</code>	Human label from the most common shared tags, falling back to the dominant indicator type.
<code>cluster_campaigns</code>	Filter to eligible indicators, build the distance matrix, run DBSCAN, assemble and sort campaigns, stamp members.

DBSCAN was chosen because the number of campaigns is unknown a priori and isolated indicators should be treated as *noise*, not forced into a cluster. Two design choices are worth stating:

- **Benign indicators are excluded** from clustering entirely, so a benign host is never reported as part of an attack campaign.
- **The distance blends two notions of “related”**: lexical similarity (DGA strings, phishing-pattern URLs and C2 IPs naturally sit apart) and tag overlap (indicators a feed labelled with the same malware family pull together even when their surface forms differ).

6.8 pipeline.py, reporting.py, cli.py

`run_pipeline` sequences the stages and returns a `PipelineResult` whose `summary()` and `to_dict()` are the single source of truth for all downstream renderings. `reporting.py` turns that into a console report, JSON, and indicator/campaign CSVs. `cli.py` exposes twelve flags (config, offline, labels, model path, saved-model reuse, no-train, corpus size, three output paths, top-N, quiet, verbose) and returns clean exit codes (2 for config errors, 1 for pipeline failure) with tracebacks hidden behind `-v`.

6.9 webapp/ — the presentation layer

A thin, read-only Flask layer runs the pipeline once at startup and serves the result. It exposes five JSON endpoints and a single-page console.

Endpoint / function	Purpose
GET /	The console: summary cards, indicators table, campaign grid, source reliability.
GET /api/summary	Run summary (counts, by-type, by-verdict, sources, training).
GET /api/indicators	Paginated, filterable indicators (verdict, type, free-text, limit $\leq$ 500, offset).
GET /api/campaigns	All campaigns with members.
GET /api/sources	Per-source reliability, ranked.
GET /healthz	Liveness probe returning status and run timestamp.
<code>create_app / build_result</code>	Wire a <code>PipelineResult</code> into a Flask app; run the pipeline at startup.
<code>static_export.render_static</code>	Render the same console to a single self-contained HTML file (data inlined, client-side filtering).

The pipeline is never exposed for remote reconfiguration: the only request parameters are pagination/filter values, and indicator text is escaped client-side before rendering. The static exporter reuses the identical template in a second mode, so the hosted GitHub Pages artifact and the live server cannot drift apart.

7 Methodology Summary

Concern	Approach	Rationale
Collection	Per-source collectors + fixture fallback	Robust to flaky feeds; reproducible
Dedup	sha256(type:value) identity	Coordination-free collapse across feeds
Feed trust	Prior blended with precision/recall/agreement	Static judgement + measured behaviour
Indicator trust	Reliability-weighted vote + consensus bonus	Corroboration rewarded, saturating
Classification	Random forest on 21 interpretable features	Explainable to analysts
Grouping	DBSCAN on composite lexical+tag distance	Unknown k ; noise tolerated

8 Experimental Setup and Results

8.1 Setup

All results come from a single deterministic offline run: `threatint -offline` against the five bundled fixtures with `random_seed = 42` and a 1,200-sample bootstrap corpus. The five feeds are Abuse.ch Feodo (domains), URLhaus (URLs), PhishTank (URLs), AlienVault OTX (mixed) and an internal honeypot (mixed).

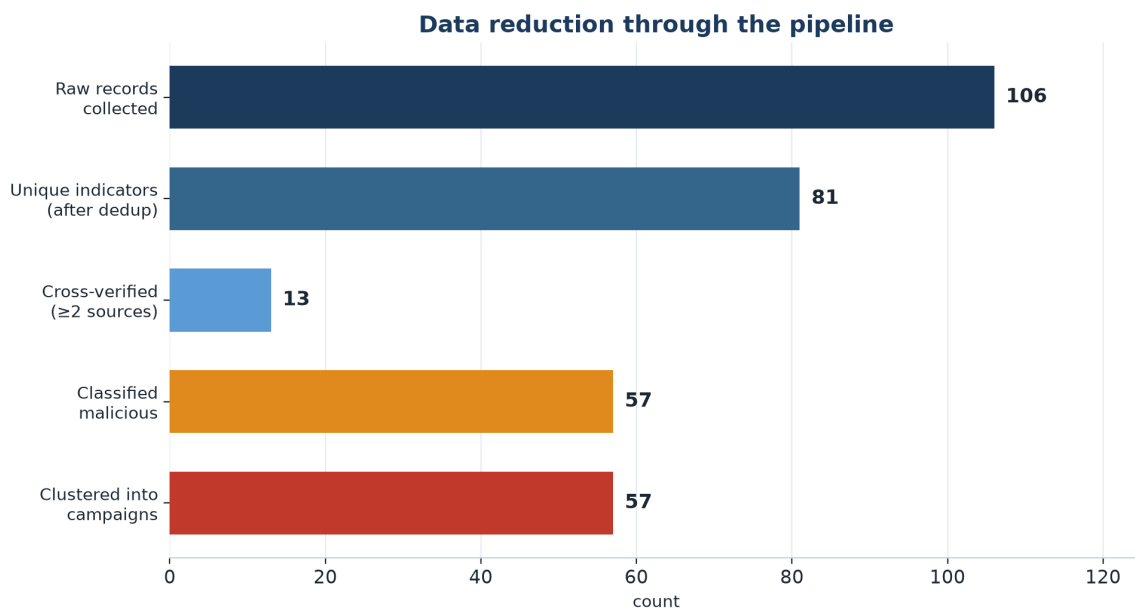


Figure 3: Data reduction through the pipeline. Deduplication collapses 106 raw records to 81 unique indicators; 57 classified malicious, of which 57 are clustered into 8 campaigns (every malicious indicator found a campaign).

8.2 Composition

Figure 4 shows the type and verdict mix. IPv4 addresses dominate (39 of 81), followed by URLs (30); the verdict split is 57 malicious to 24 benign, with no indicators in the ambiguous band.

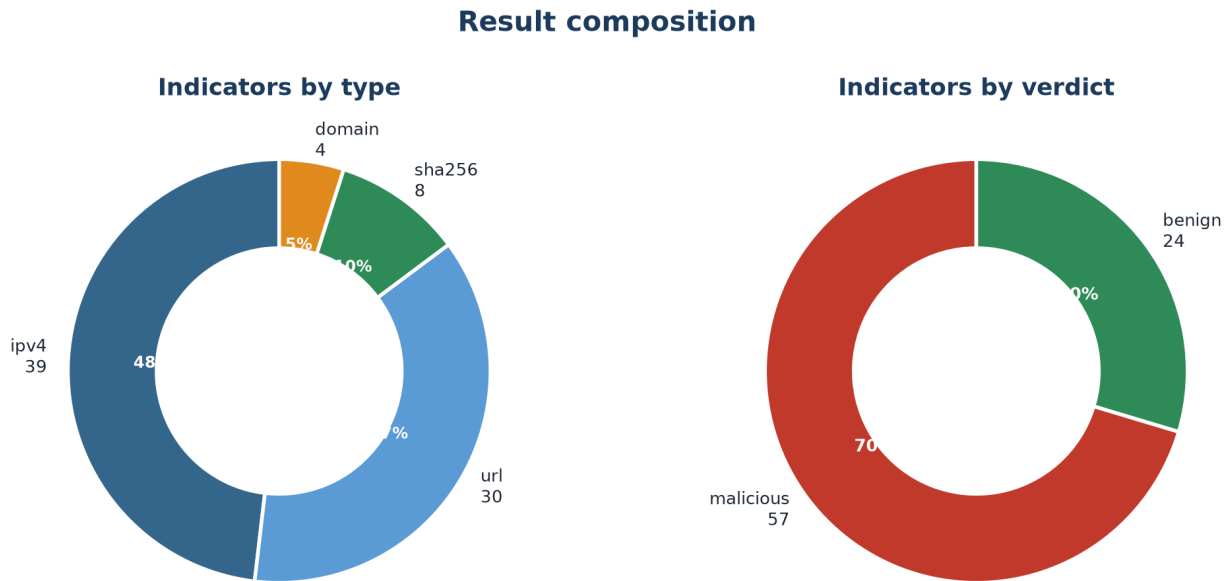


Figure 4: Indicator composition by type (left) and verdict (right).

8.3 Source reliability

The table and figure below show how each feed’s static prior is adjusted by measured behaviour. The internal honeypot has the lowest prior (0.70) but the highest recall (1.00) and the best precision (0.565), so its dynamic evidence pulls it well above its prior. Conversely Feodo keeps a high prior (0.90) and ends with the top score (0.692) despite low precision — the intended effect of letting a strong prior resist a noisy batch.

Source	prior	prec.	recall	agree.	vol.	score
abusech_feodo	0.90	0.217	0.385	0.049	23	0.692
urlhaus	0.85	0.364	0.615	0.117	22	0.705
phishtank	0.80	0.333	0.462	0.108	18	0.671
alienvault_otx	0.75	0.300	0.462	0.100	20	0.621
internal_honeypot	0.70	0.565	1.000	0.168	23	0.656

8.4 Classifier performance

The classifier was evaluated on a held-out 25% stratified test set. The confusion matrix and ROC curve below were produced by replicating the exact seeded split, so they match the reported metrics precisely.

Metric	Accuracy	Precision	Recall	F1
Value	0.860	0.915	0.793	0.850

The precision–recall asymmetry is deliberate and desirable for this domain: precision (0.915) exceeds recall (0.793), meaning the model is conservative about calling something malicious. In

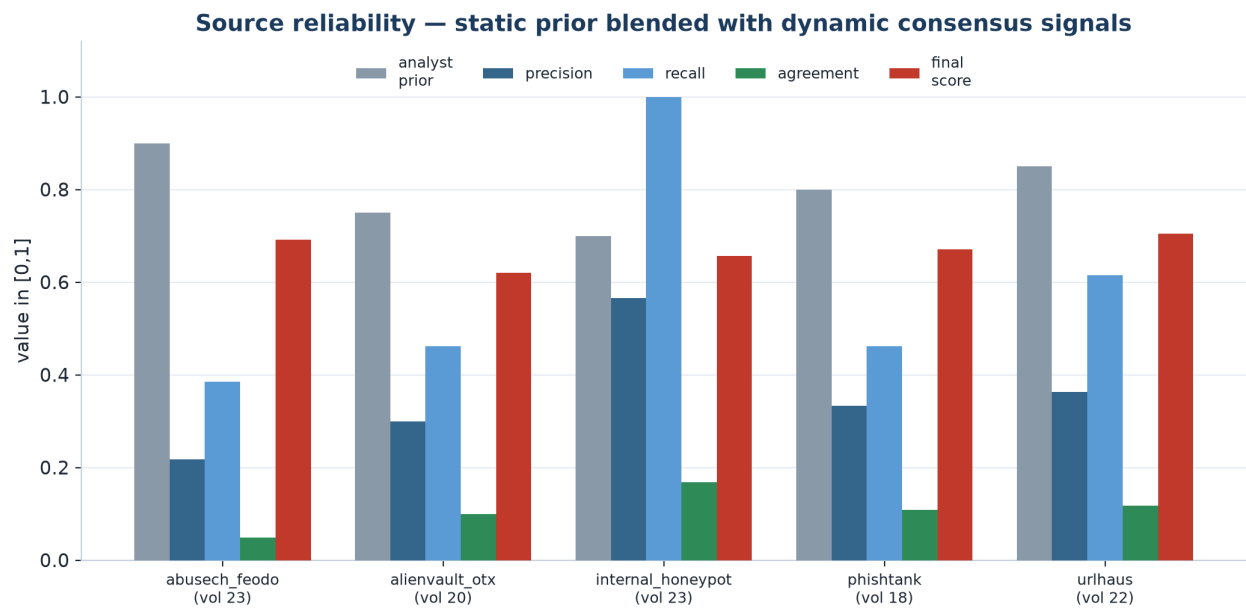


Figure 5: Source reliability. Prior, precision, recall, agreement and the blended final score for each feed.

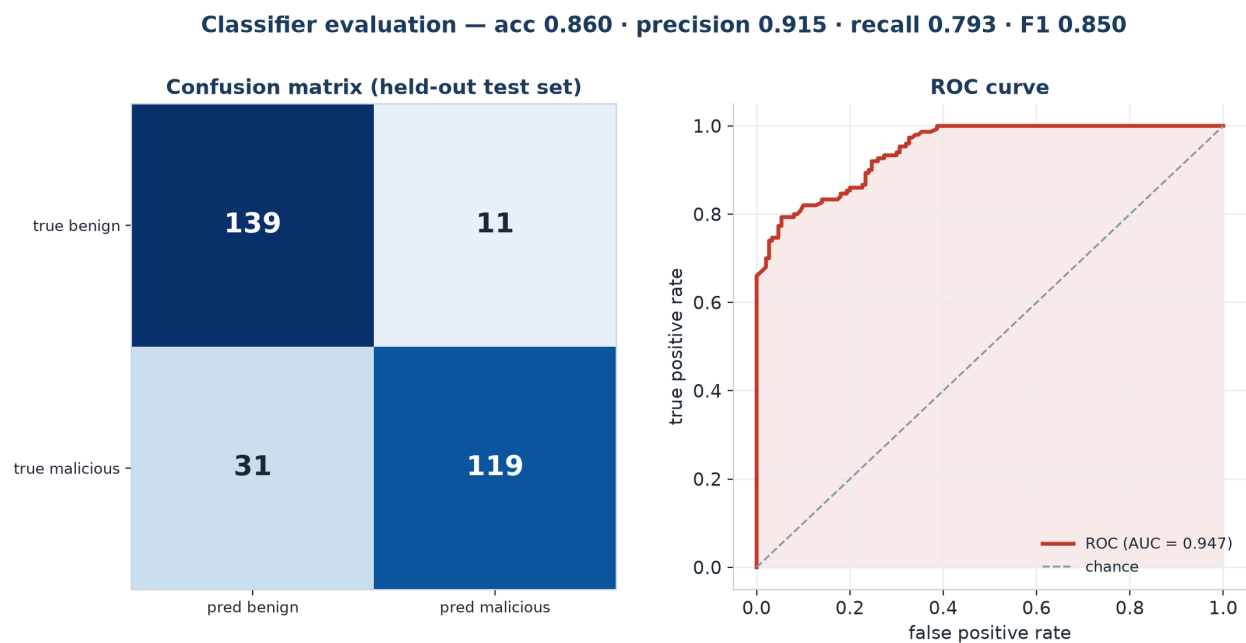


Figure 6: Classifier evaluation. Left: confusion matrix on the held-out test set. Right: ROC curve (AUC 0.946).

threat intelligence a false positive (wasting analyst time, or worse, blocking a legitimate host) is costlier than a miss, so this bias is the right one. ROC AUC of 0.946 indicates strong separability overall.

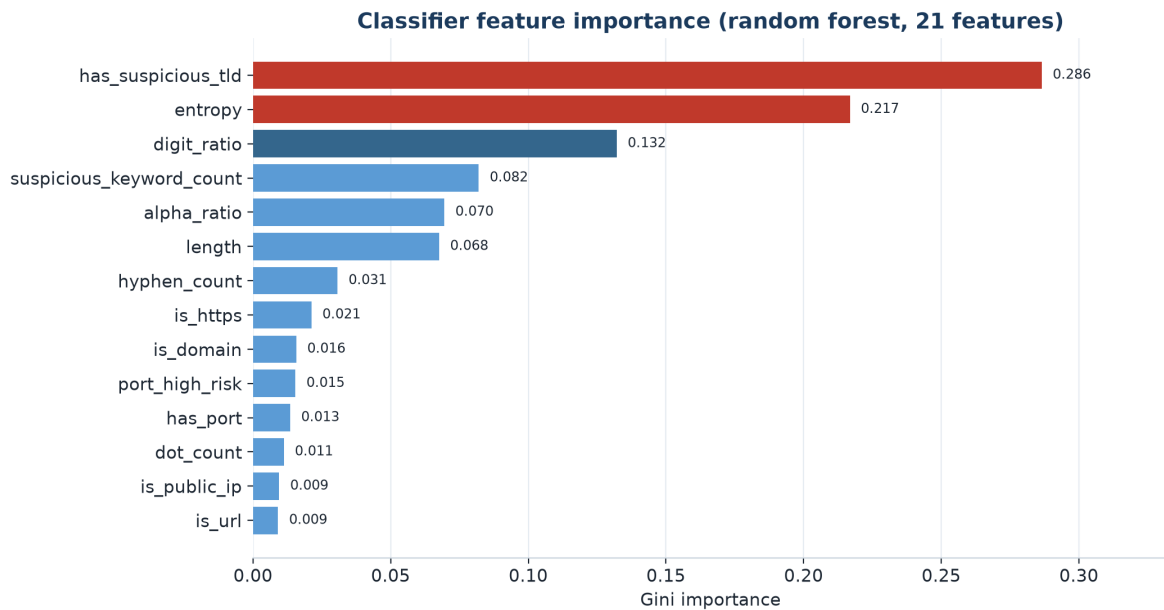


Figure 7: Random-forest feature importance. The model relies most on `has_suspicious_tld` (0.287), `entropy` (0.217) and `digit_ratio` (0.132) — exactly the signals an analyst would use.

Feature importance (Figure 7) confirms interpretability: the top three signals are the suspicious-TLD flag, string entropy (DGA detection) and digit ratio — human-legible heuristics, not opaque interactions.

8.5 Campaign clustering

DBSCAN discovered 8 campaigns covering all 57 malicious indicators, with zero noise points. The campaigns are coherent and map to recognisable threat families.

Campaign	Label	Size	$\bar{p}$	$\bar{r}$	Shared tags
camp-807bd8b856	malware	14	0.99	0.82	malware, payload
camp-1685e9e43e	credential	12	0.99	0.84	credential, phishing
camp-5a786a8527	botnet	10	0.68	0.90	botnet, c2, trickbot
camp-8c3164533c	exploit	8	0.98	0.88	exploit, loader, payload, ...
camp-84f67a7ae8	bruteforce	5	0.66	0.60	bruteforce, honeypot
camp-3d0ba5cd49	botnet	3	0.74	0.90	botnet, c2, emotet, apt
camp-904da0dd19	apt	3	0.63	0.70	apt, malware
camp-3130c44ab8	scanner	2	0.61	0.60	scanner

Campaign IDs are reproducible

Because a campaign’s ID is a hash of its members’ stable indicator IDs, the same input data always produces the same campaign identifiers. This makes it possible to track a campaign’s evolution across runs — an important property for operational use that a naive random cluster label would not provide.

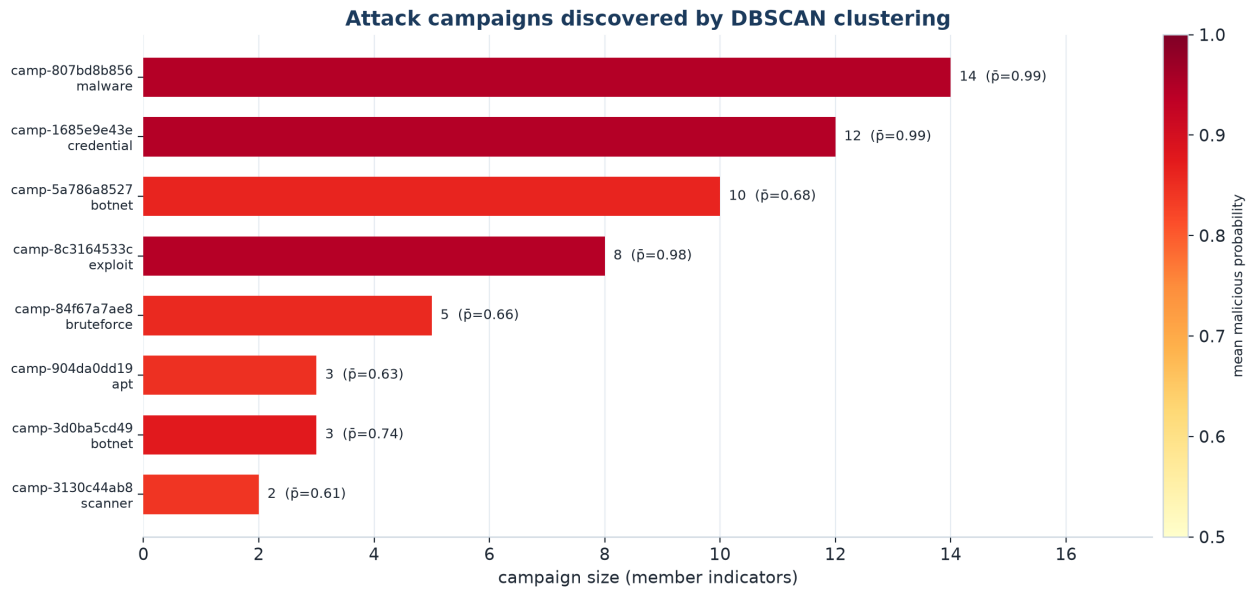


Figure 8: Campaigns discovered by DBSCAN, sized by membership and coloured by mean malicious probability. Labels derive from the most common shared tags.

8.6 Per-feed contribution

Finally, Figure 9 shows how much each feed contributes and how much survives deduplication — the practical measure of feed overlap.

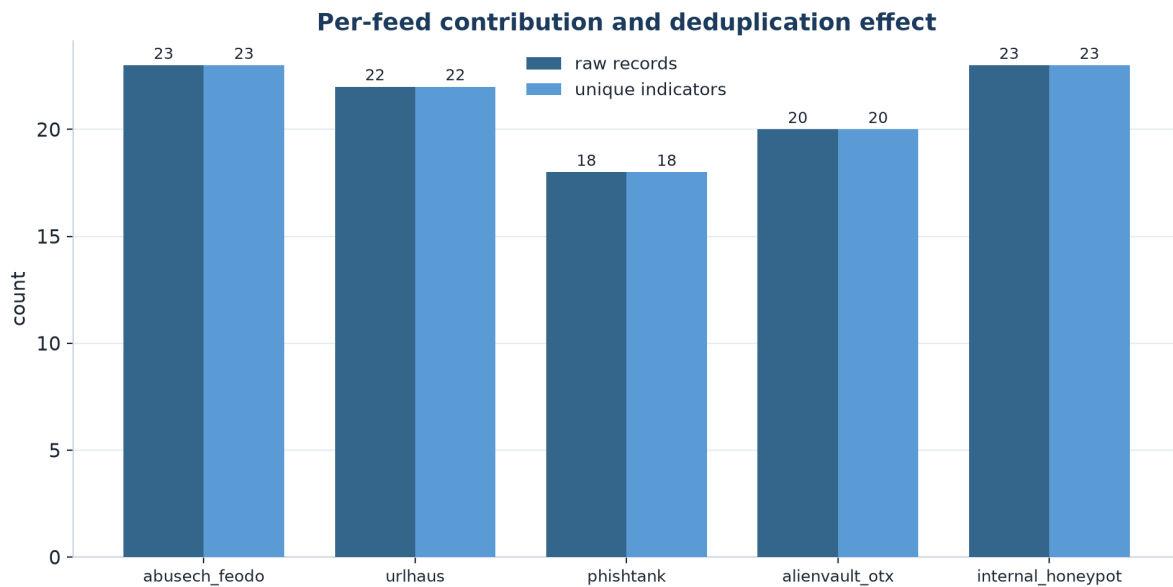


Figure 9: Raw records versus unique indicators contributed per feed. The gap is the overlap that manual triage would otherwise reconcile by hand.

9 Interfaces, Deployment and Reproducibility

9.1 Command-line interface

```
1 # Deterministic offline run (the reference run in this report)
2 threatint --offline
```

```

3
4 # Live feeds, with per-source fixture fallback on any failure
5 threatint
6
7 # Train on analyst-reviewed labels and persist the model
8 threatint --labels labels.csv --model-path artifacts/classifier.joblib
9
10 # Reuse a saved model without retraining; export machine-readable output
11 threatint --use-saved-model --out-json out.json --out-indicators iocs.csv

```

9.2 Web dashboard and static export

```

1 threatint-web --offline --port 12000      # live Flask console + JSON API
2 threatint-static --offline --out dist/index.html # single-file HTML export

```

The dashboard is a read-only “operations console”: an indicators table with probability, verdict, type, source count, reliability and campaign, filterable and paginated; a campaign grid; and a source-reliability ranking. All indicator values are defanged before display.

9.3 Deployment paths

Three deployment paths ship in the repository, all exercised during development:

Path	Mechanism
GitHub Pages	publish-dashboard.yml renders the static console and publishes it on push to main. No server, no secrets.
Docker	Dockerfile installs the package, bakes a static dashboard into the image, and serves the live console on port 12000.
CI	ci.yml runs pyflakes and the full pytest suite on Python 3.10 and 3.12.

9.4 Testing

The suite comprises **60 tests** across eight modules (models, normalize, features, classifier, clustering, verification, pipeline, webapp), all running offline and deterministically. Lint is clean under pyflakes. The tests assert real behaviour, not mocks: the end-to-end test runs the whole pipeline and checks that a deliberately shared indicator is reported by at least three sources.

10 Security Considerations

Because the system handles indicators that are themselves adversarial strings, several precautions matter:

- **Defanging on display.** The dashboard renders every indicator defanged (`hxxp://...[.]`) so a live IOC is never a clickable link.
- **Output escaping.** Indicator text is escaped client-side before insertion into the DOM, preventing stored-XSS via a malicious feed.
- **No remote reconfiguration.** The web layer accepts only pagination and filter parameters; the pipeline cannot be steered remotely.
- **Least-privilege API keys.** Live feed credentials are read from environment variables named in config, never hard-coded.

- **Bounded inputs.** API page size is clamped (≤ 500); offset is non-negative; malformed integers fall back to safe defaults.
- **Provenance retained.** Raw payloads are preserved, so every downstream decision can be audited back to its source record.

11 Limitations

Honest scoping matters more than a flattering one. The principal limitations:

1. **The default model trains on synthetic data.** The reported metrics characterise the pipeline, not production accuracy. Real use requires analyst-reviewed labels via `-labels`; the interface is ready and the report always records `data_source`.
2. **The reference run is small and offline.** 81 indicators is enough to exercise every stage, not to characterise feed-scale behaviour. A live validation did ingest 32,289 URLhaus records and 106,407 records in total, demonstrating scale, but that run is not the basis of these metrics.
3. **Agreement is corpus-relative.** Precision/recall are measured against a consensus that itself assumes majority-correctness; a systematically wrong majority would propagate.
4. **No temporal decay.** Features include indicator age, but source reliability is not yet time-weighted; a feed that was good last year scores the same as one that is good today.
5. **Clustering distance is heuristic.** The lexical/tag blend and its 0.6/0.4 weighting are principled but not learned or tuned against labelled campaigns.

12 Future Work

- **Analyst-labelled training loop:** feed confirmed/denied verdicts back as labels, with periodic retraining and drift monitoring.
- **Temporal source weighting:** exponentially decay historical agreement so reliability tracks a feed's current behaviour.
- **Graph-based campaign linking:** model shared infrastructure (registrar, ASN, certificate, hosting) as a graph to link campaigns beyond lexical/tag similarity.
- **Enrichment:** passive-DNS, WHOIS and certificate-transparency lookups to strengthen weak signals for low-reliability indicators.
- **Explainability surface:** expose per-indicator feature contributions (e.g. SHAP) directly in the dashboard so an analyst sees *why* the model scored something.
- **Feed-scale evaluation:** rerun the full metrics against a large, labelled, live corpus and publish calibration curves.

13 Conclusion

ThreatInt demonstrates that the mechanical layer of threat triage — collection, reconciliation, scoring and grouping — can be automated into a small, reproducible, well-tested system. Its distinguishing qualities are not exotic algorithms but sound engineering: computed (not assumed) trust, network-optional determinism, interpretable features, and honest labelling of synthetic data. On the reference run it turns 106 records from five disagreeing feeds into 81 deduplicated indicators, 57 classified malicious with 0.946 AUC, and 8 coherent attack campaigns with reproducible identifiers — while running offline, in CI, and on a laptop.

The system is deliberately a foundation. The path to production is clear and mostly about data, not architecture: point `-labels` at analyst-reviewed ground truth, widen the corpus, and add temporal and graph-based enrichment. The pipeline is built so those upgrades drop in without redesign.

A Configuration Reference

```

1 pipeline:
2   min_sources_for_verification: 2      # consensus threshold m
3   malicious_threshold: 0.5            # verdict cutoff theta
4   min_reliability_for_clustering: 0.35 # clustering eligibility floor
5   random_seed: 42                    # determinism
6
7 sources:                             # name / kind / prior / url / fixture
8   - abusech_feodo    (domain, 0.90)   # feodotracker blocklist
9   - urlhaus          (url,    0.85)   # urlhaus recent URLs
10  - phishtank         (url,    0.80)   # phishtank online-valid
11  - alienvault_otx    (mixed, 0.75)   # OTX pulses
12  - internal_honeypot (mixed, 0.70)   # local captures (no URL)
13
14 features:    suspicious_tlds, suspicious_keywords, high_risk_ports
15 ml:          model, n_estimators, max_depth, test_size, model_path
16 clustering:  eps: 0.22, min_samples: 2 # DBSCAN parameters

```

B CLI Reference

Flag	Purpose
<code>-config PATH</code>	Config file (defaults to repo config).
<code>-offline</code>	Use bundled fixtures only; never touch the network.
<code>-labels PATH</code>	Analyst labels CSV (indicator, label) for supervised training.
<code>-model-path PATH</code>	Where to save/load the trained classifier.
<code>-use-saved-model</code>	Load a saved model instead of retraining.
<code>-no-train</code>	Skip training (verdicts stay UNKNOWN).
<code>-corpus-size N</code>	Synthetic bootstrap corpus size (default 1200).
<code>-out-json /</code>	Machine-readable exports.
<code>-out-indicators /</code>	
<code>-out-campaigns</code>	
<code>-top N</code>	Rows in the console report.
<code>-quiet / -v</code>	Minimal output / debug logging.

C Reproducing This Report

```

1 python -m venv .venv && .venv/bin/pip install -e ".[dev,web]"
2 .venv/bin/python -m pytest -q                               # 60 tests
3 .venv/bin/threatint --offline                               # the reference run
4 .venv/bin/python report/make_figures.py                     # regenerate every figure
5 .venv/bin/python report/build_report.py                     # compile this PDF

```